Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Военный факультет

Кафедра электронных вычислительных машин

ОТЧЕТ
по лабораторной работе № 1
по дисциплине «Жизненный цикл разработки программного обеспечения»
на тему

ИНТЕГРИРОВАННАЯ СРЕДА РАЗРАБОТКИ ПО

Вариант 14

Выполнил студент гр. 434701 В. К. Мурачев

Проверил О. М. Внук

Минск 2026

1 ЦЕЛЬ РАБОТЫ

Целью лабораторной работы является:

– изучить методы работы с одной из выбранных сред разработки при условии, что используется только объектно-ориентированный язык программирования;

– научиться разрабатывать в выбранной среде приложение с графическим интерфейсом;

– изучить паттерн MVC с активной и пассивной моделью.

Работа выполняется по варианту 14. Реализован основной уровень задания – каркас MVC с активной моделью.

2 ЗАДАНИЕ К ЛАБОРАТОРНОЙ РАБОТЕ

Порядок выполнения работы включает изучение теоретических сведений, ответы на контрольные вопросы, выполнение задания по варианту и оформление отчета. Задание состоит из следующих пунктов:

1) Получить вариант у преподавателя. Выбрать объектно-ориентированный язык программирования и среду разработки, согласовать их с преподавателем.

2) Установить и настроить программное обеспечение. Ознакомиться с его функциональными возможностями и средствами отладки, привести примеры работы с функционалом и продемонстрировать работу отладчика.

3) Создать пустой проект под выданный вариант задания. Установить расширения для работы с SysML v2 и UML. Результат отобразить в отчете.

4) Разработать и описать в отчете каркас MVC с выданной по варианту моделью.

2.1 Задание по варианту 14

Разработать программу «Калькулятор Жизни». Программа по введенному количеству лет и количеству часов в день, проведенных за компьютером, должна вычислить, сколько человек просидел за компьютером часов и суток, каков пробег курсора в километрах, сколько раз примерно человек кликнул мышью и сколько раз нажал на клавиатуру (используя статистические данные из интернета). На главной форме размещается кнопка «Ввести данные», по нажатию на которую открывается дополнительное окно для ввода данных. Последние введенные данные запоминаются в главной форме и восстанавливаются при повторном вводе. После этого на главной форме отображается результат вычислений. Если введены некорректные данные, выводится сообщение об ошибке. Требуется добавить каркас MVC с активной моделью.

Из формулировки варианта выделены следующие требования к программе:

– ввод количества лет и количества часов в день, проведенных за компьютером;

– кнопка «Ввести данные» на главной форме;

– отдельное окно для ввода данных;

– запоминание последних введенных данных и их восстановление при повторном вводе;

– расчет времени за компьютером, пробега курсора, количества кликов и нажатий клавиш;

– отображение результата на главной форме;

– сообщение об ошибке при некорректных данных;

– каркас MVC с активной моделью.

3 ВЫБОР ЯЗЫКА И СРЕДЫ РАЗРАБОТКИ

В качестве языка программирования выбран объектно-ориентированный язык C++ (стандарт C++17), в качестве библиотеки графического интерфейса – Qt 6 (модуль Qt Widgets), в качестве среды разработки – Microsoft Visual Studio Community 2026.

Выбор обоснован тем, что библиотека Qt предоставляет готовые классы окон и диалогов, а ее механизм сигналов и слотов реализует шаблон «Наблюдатель», который и требуется для активной модели: модель рассылает сигнал об изменении состояния, а представление принимает его своим слотом. Среда Visual Studio работает с проектами CMake без дополнительной настройки, содержит компилятор MSVC, систему сборки Ninja, отладчик и диспетчер расширений, через который устанавливаются расширения для работы с UML.

Состав инструментального окружения приведен в таблице 1.

Таблица 1 – Состав инструментального окружения

Компонент	**Версия**	**Назначение**
Операционная система	Windows 11	Платформа разработки и запуска
Язык программирования	C++17	Объектно-ориентированный язык реализации
Среда разработки	Visual Studio Community 2026	Редактирование, сборка, запуск и отладка проекта
Компилятор	MSVC 19.51	Компиляция и компоновка исходных текстов
Библиотека интерфейса	Qt 6.7.3 (msvc2019_64)	Классы окон и диалогов, сигналы и слоты
Система сборки	CMake 4.3.1, Ninja	Описание и выполнение сборки проекта
Отладчик	Отладчик Visual Studio	Точки останова, пошаговое выполнение, просмотр переменных
Расширение для UML	PlantUML Editor 1.1.133	Редактирование и отрисовка диаграмм UML

4 УСТАНОВКА И НАСТРОЙКА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

4.1 Установка среды разработки и библиотеки Qt

Среда Visual Studio Community 2026 установлена с рабочей нагрузкой «Разработка классических приложений на C++». В состав нагрузки входят компилятор MSVC, системы сборки CMake и Ninja, отладчик и средства работы с проектами CMake, поэтому отдельная установка этих компонентов не потребовалась.

Библиотека Qt 6.7.3 установлена комплектом msvc2019_64, собранным тем же компилятором MSVC, что и проект. В состав комплекта входят модуль Qt Widgets и утилита windeployqt, которая копирует необходимые динамические библиотеки рядом с исполняемым файлом, что позволяет запускать программу вне среды.

Проект открывается в среде как папка с CMake-проектом (Файл → Открыть → Папка), после чего среда сама выполняет конфигурирование и предлагает цель запуска. Главное окно среды с открытым проектом приведено на рисунке 1.

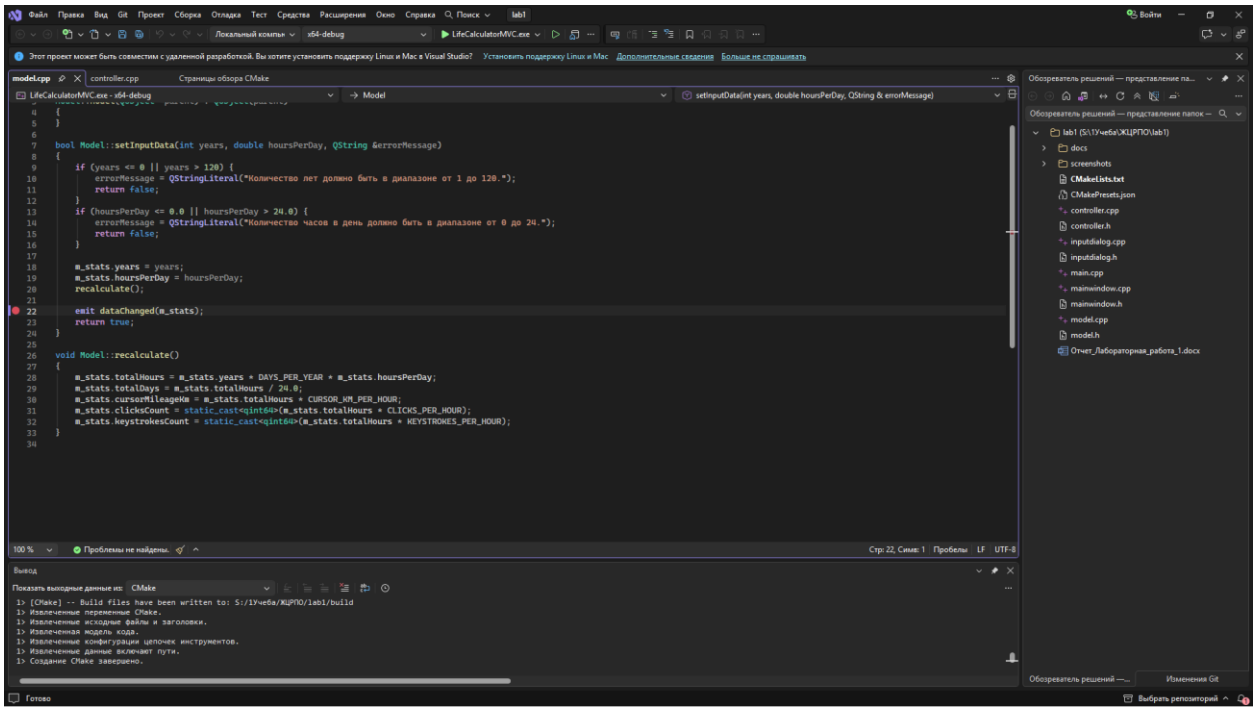


Рисунок 1 – Главное окно Visual Studio с открытым проектом

4.2 Настройка сборки проекта

Параметры сборки вынесены в файл CMakePresets.json (приложение А): конфигурация x64-debug использует генератор Ninja, тип сборки Debug и переменную CMAKE_PREFIX_PATH, указывающую на комплект Qt. Благодаря этому одна и та же конфигурация используется и средой, и командной строкой: cmake --preset x64-debug и cmake --build --preset x64-debug.

Исходные тексты содержат строковые литералы на русском языке, поэтому в файл CMakeLists.txt добавлен флаг /utf-8 для компилятора MSVC. Без него компилятор считает файлы записанными в кодировке системы, и сообщения программы выводятся неразборчиво.

Ход конфигурирования и сборки отображается в окне «Вывод» (рисунок 2). Сборка выполняется сочетанием клавиш Ctrl+Shift+B, запуск без отладки – Ctrl+F5.



Рисунок 2 – Окно «Вывод» с результатом сборки проекта

4.3 Функциональные возможности среды

Основные компоненты Visual Studio и их назначение приведены в таблице 2.

Таблица 2 – Основные компоненты среды Visual Studio

Компонент	Назначение и основные возможности
Редактор кода	Подсветка синтаксиса, автодополнение IntelliSense, подсказки по сигнатурам, сворачивание блоков кода
Обозреватель решений	Просмотр структуры проекта в режиме папки, открытие и создание файлов
Поддержка CMake	Конфигурирование проекта по CMakeLists.txt и CMakePresets.json, выбор конфигурации сборки
Окно «Вывод»	Журнал конфигурирования CMake и сборки, сообщения компилятора и компоновщика
Окно «Список ошибок»	Ошибки и предупреждения с переходом к строке исходного текста
Отладчик	Точки останова, пошаговое выполнение, окна «Локальные», «Видимые», «Контрольные значения» и «Стек вызовов»
Диспетчер расширений	Поиск, установка и удаление расширений из Visual Studio Marketplace
Средства рефакторинга	Переименование символа, извлечение метода, быстрые действия по подсказкам анализатора

В ходе работы использованы следующие возможности среды:

– навигация по коду: переход к определению (F12), поиск по символу (Ctrl+T), переключение между заголовочным файлом и файлом реализации;

– автодополнение IntelliSense и подсказки по сигнатурам методов Qt;

– переименование символов и другие быстрые действия редактора;

– сборка (Ctrl+Shift+B) и запуск проекта, просмотр ошибок компиляции в окнах «Список ошибок» и «Вывод»;

– установка расширений из Visual Studio Marketplace.

4.4 Средства отладки

Отладчик среды запускается клавишей F5. Основные приемы работы:

– установка и снятие точки останова – щелчок по полю слева от номера строки или F9;

– шаг с обходом функции – F10, шаг с заходом в функцию – F11, выход из функции – Shift+F11, продолжение выполнения – F5;

– окна «Локальные» и «Видимые» показывают значения переменных текущего кадра стека, в том числе поля структур и объектов Qt;

– окно «Контрольные значения» позволяет следить за произвольными выражениями;

– окно «Стек вызовов» показывает цепочку вызовов, приведшую к текущей точке;

– в свойствах точки останова задаются условие и число пропусков.

Для демонстрации отладчика точка останова установлена в методе Controller::onEnterDataRequested() на строке передачи данных модели (рисунок 3). В окне «Локальные» видно, что текст полей окна ввода успешно преобразован в числа, а окно «Стек вызовов» показывает, что слот вызван из сигнала кнопки через механизм сигналов и слотов Qt.

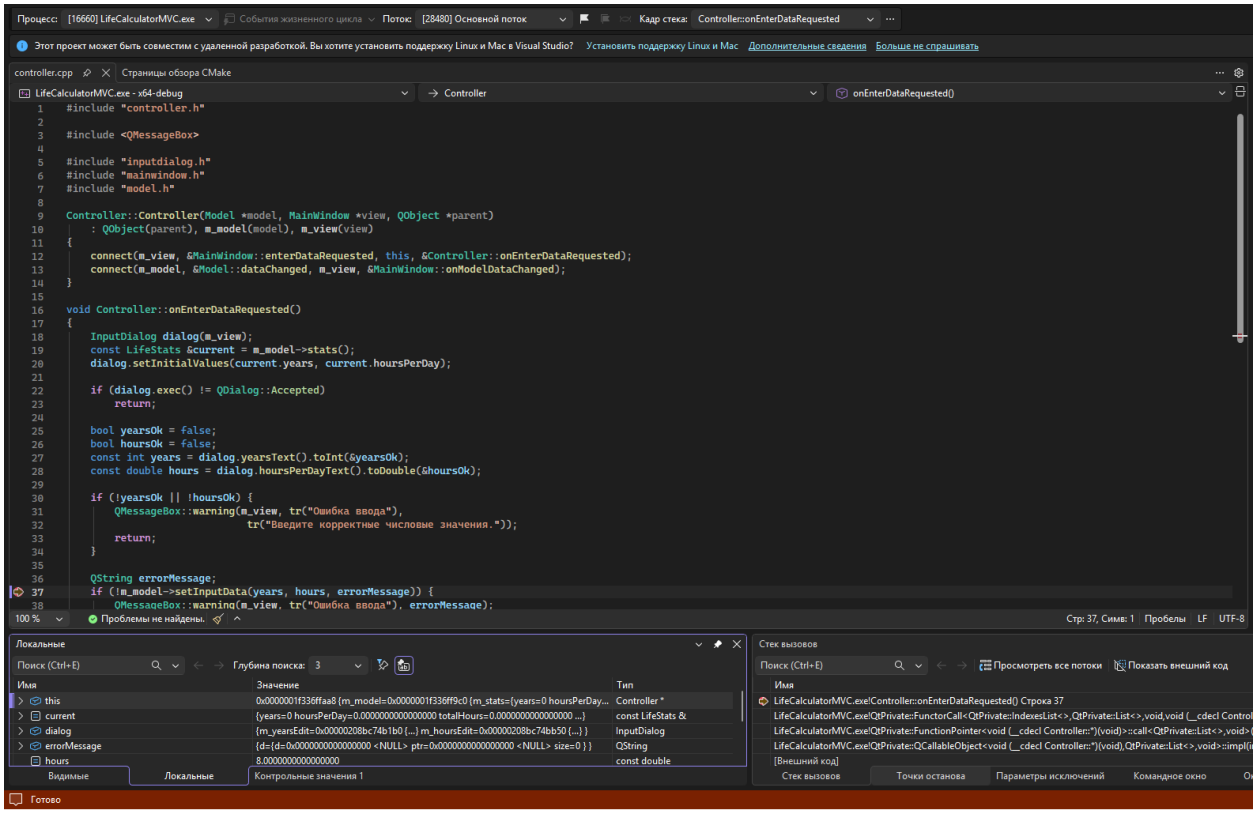


Рисунок 3 – Остановка отладчика в методе Controller::onEnterDataRequested()

Вторая точка останова установлена в методе Model::setInputData() на строке рассылки сигнала. К этому моменту модель уже проверила данные и выполнила пересчет, поэтому в окне «Видимые» структура m_stats содержит рассчитанные значения (рисунок 4). Это позволяет убедиться в правильности расчета до того, как представление получит уведомление.

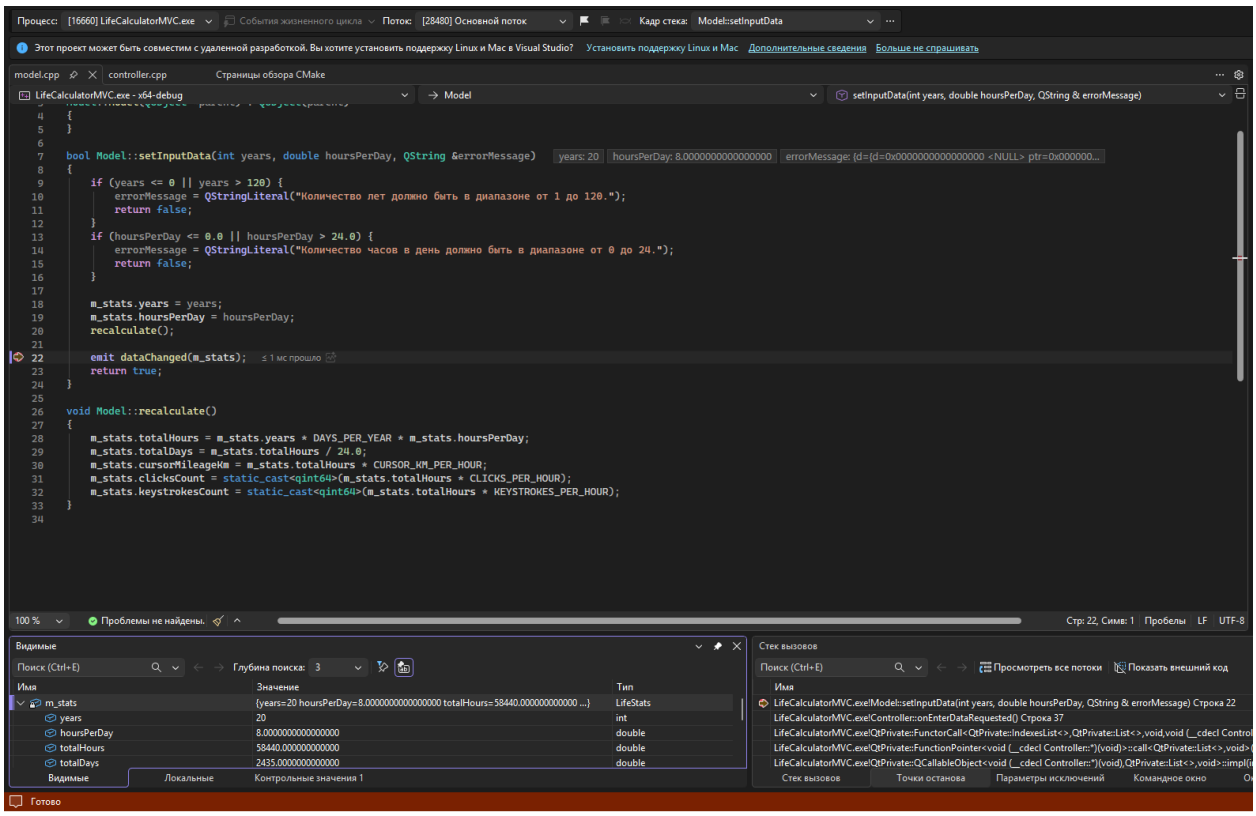


Рисунок 4 – Значения структуры m_stats при остановке в методе Model::setInputData()

5 СОЗДАНИЕ ПРОЕКТА И РАСШИРЕНИЯ ДЛЯ UML И SYSML V2

5.1 Создание проекта

Проект создан как проект CMake. В каталоге проекта создан файл CMakeLists.txt с описанием цели LifeCalculatorMVC, списком исходных файлов, поиском пакета Qt функцией find_package() и подключением модуля Qt6::Widgets. Автоматическая обработка макроса Q_OBJECT включается параметром CMAKE_AUTOMOC – без него компоновщик не найдет код, который Qt генерирует для сигналов и слотов.

Структура проекта приведена в таблице 3.

Таблица 3 – Структура проекта

Каталог / файл	Содержимое
CMakeLists.txt	Описание проекта для CMake: цель, исходные файлы, поиск Qt, флаг /utf-8
CMakePresets.json	Конфигурация x64-debug: генератор Ninja, путь к комплекту Qt
main.cpp	Точка входа: создание модели, представления и контроллера, запуск цикла событий
model.h, model.cpp	Слой Model: структура LifeStats и активная модель Model
mainwindow.h, mainwindow.cpp	Слой View: главная форма и слот обновления результатов
inputdialog.h, inputdialog.cpp	Слой View: модальное окно ввода данных
controller.h, controller.cpp	Слой Controller: обработка действия пользователя
docs/	Диаграммы в нотации PlantUML и модель системы в нотации SysML v2
build/	Каталог сборки, создается средой при конфигурировании

Полные исходные тексты программы приведены в приложении А.

5.2 Расширения для работы с UML и SysML v2

Примеры расширений в задании приведены для MS Visual Studio Code, поэтому для выбранной среды подобраны аналоги в Visual Studio Marketplace.

Для работы с UML установлено расширение PlantUML Editor версии 1.1.133 (рисунок 5). Расширение добавляет в среду редактор файлов с расширением .mdpuml, в которых текстовое описание диаграммы в нотации

PlantUML соседствует с пояснениями в разметке Markdown, и отображает готовую диаграмму рядом с исходным текстом (рисунок 6).

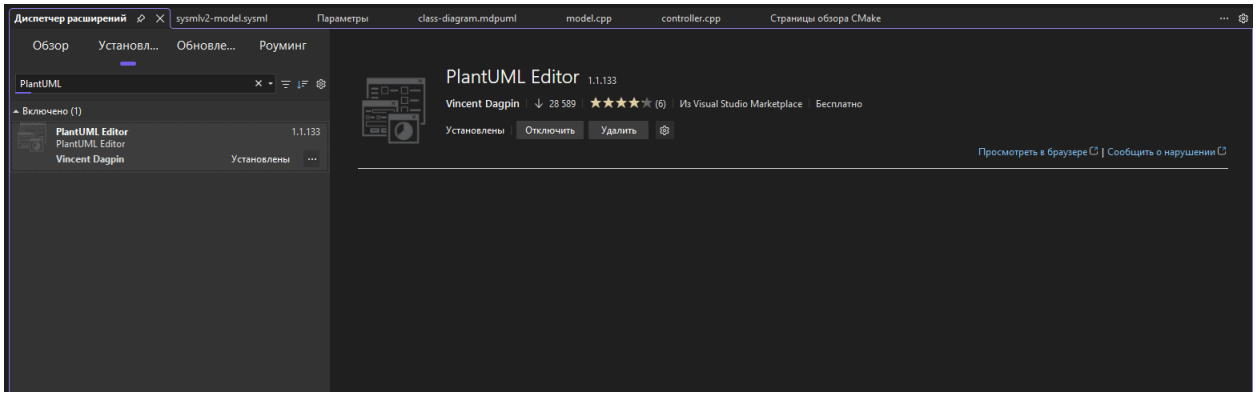


Рисунок 5 – Установленное расширение PlantUML Editor в диспетчере расширений

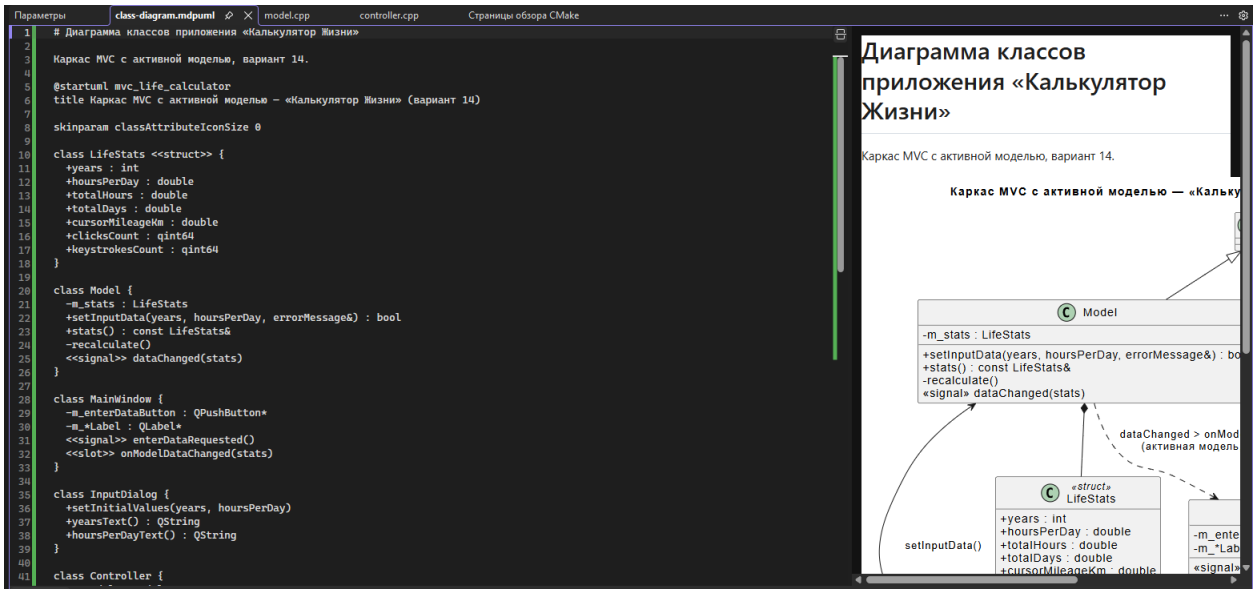


Рисунок 6 – Диаграмма классов в редакторе расширения PlantUML Editor

Расширения для работы с SysML v2 в Visual Studio Marketplace отсутствуют: поиск по запросу sysml не дает результатов (рисунок 7). Поэтому модель системы описана текстом в файле docs/sysmlv2-model.sysml и редактируется в текстовом редакторе среды (рисунок 8). Для других сред аналогами являются расширение SysML v2 для Visual Studio Code и пилотная реализация SysML v2 на базе Eclipse.

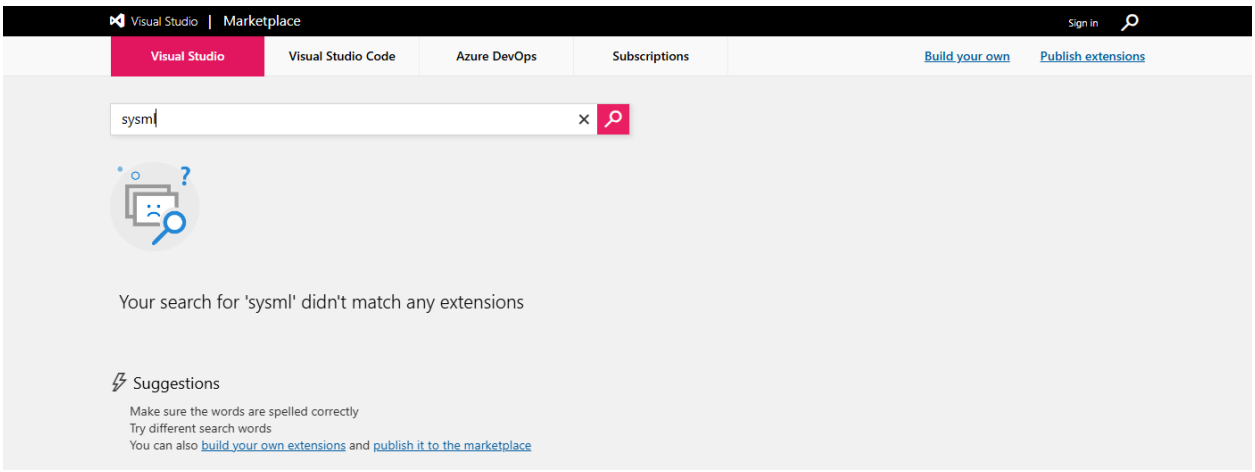


Рисунок 7 – Результат поиска расширений SysML v2 в Visual Studio Marketplace

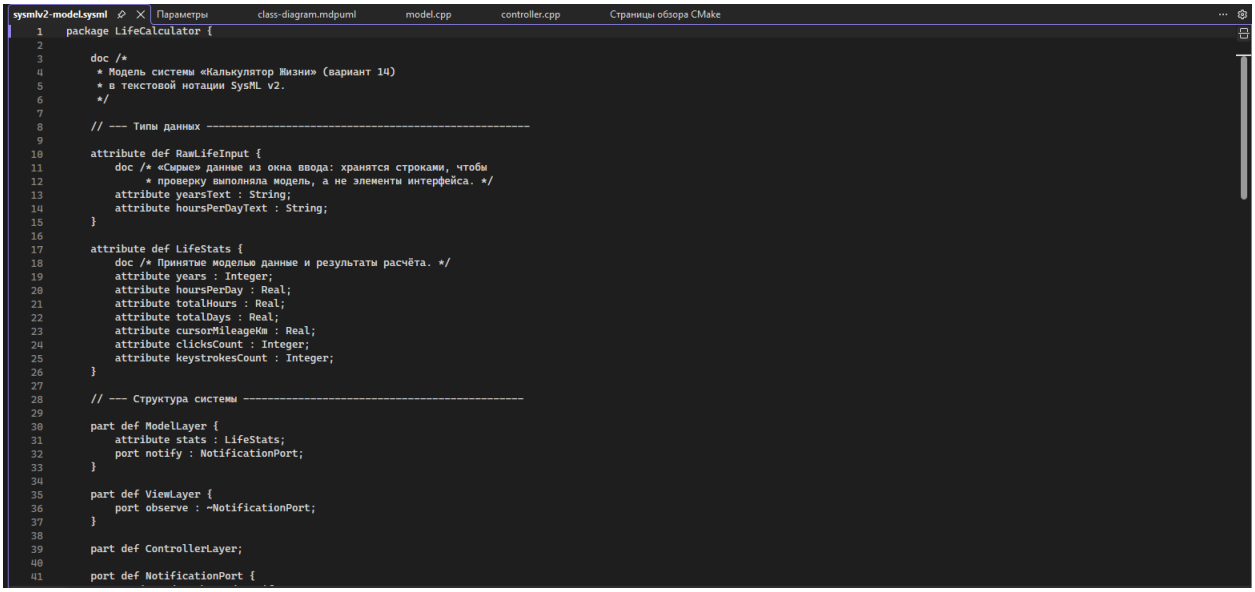


Рисунок 8 – Модель системы в нотации SysML v2 в редакторе среды

5.3 UML-модель приложения

Диаграмма классов приложения построена в нотации UML и приведена на рисунке 9. Текстовое описание диаграммы хранится в файле docs/class-diagram.puml проекта.

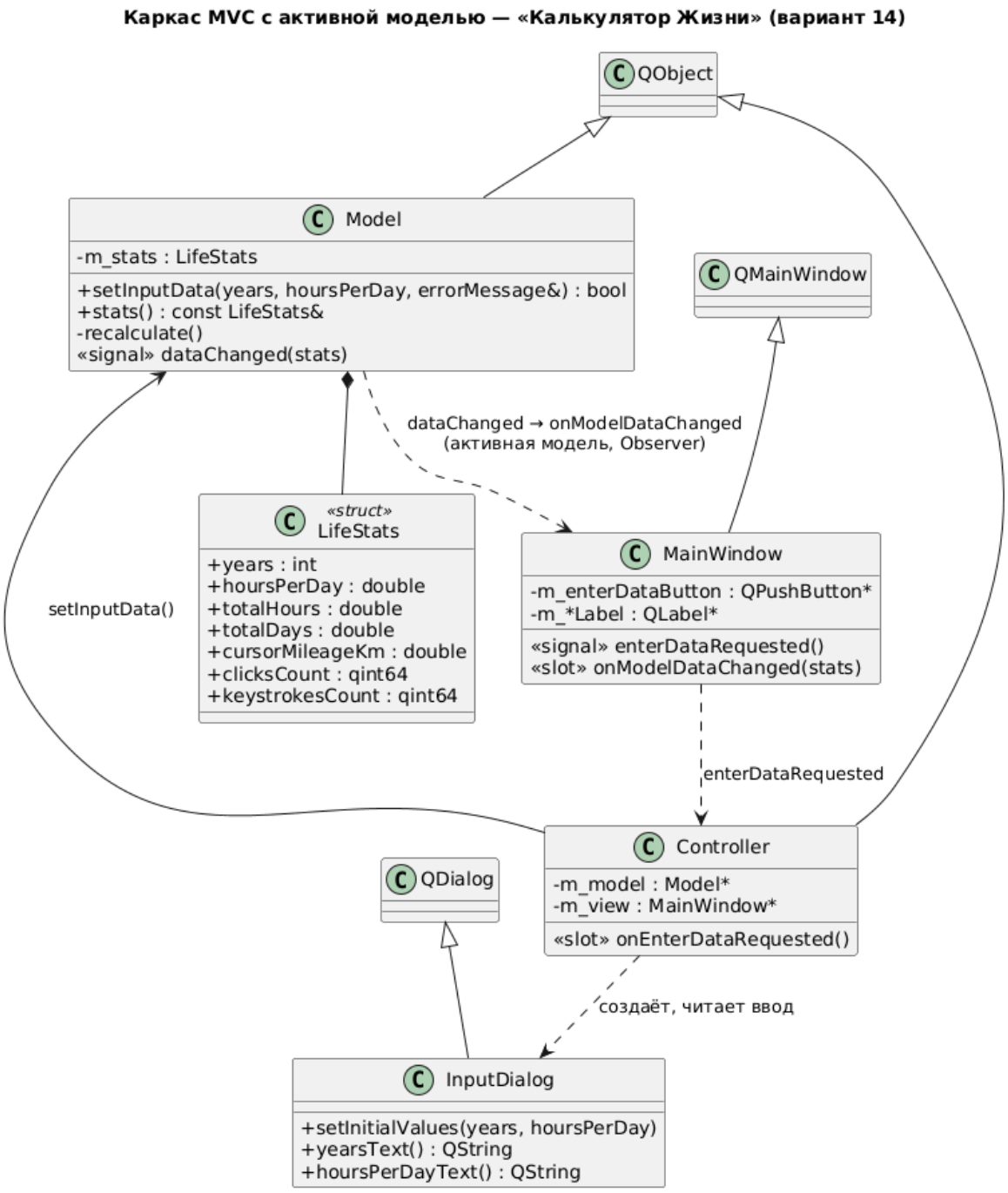


Рисунок 9 – Диаграмма классов приложения

На диаграмме видно разделение классов по слоям: MainWindow и InputDialog относятся к представлению, Controller – к контроллеру, класс Model вместе со структурой LifeStats – к модели. Сплошными линиями показаны наследование от классов Qt и состав, пунктирными – направления взаимодействия: представление сообщает контроллеру о действии пользователя, контроллер передает данные модели, модель уведомляет представление об изменении состояния.

5.4 Модель системы в нотации SysML v2

Модель системы описана в файле docs/sysmlv2-model.sysml (приложение Б). В ней определены:

– типы данных RawLifeInput и LifeStats (attribute def): «сырые» данные окна ввода и состояние модели с результатами расчета;

– структура системы Application из трех частей model, view и controller (part def) и связь между моделью и представлением через порт NotificationPort, по которому передается уведомление об изменении состояния;

– поведение CalculateLifeStats (action def): запоминание ввода, проверка, пересчет показателей и оповещение представления;

– требования варианта R1–R6 (requirement def) и их удовлетворение частью variant14 (satisfy).

6 РАЗРАБОТКА КАРКАСА MVC С АКТИВНОЙ МОДЕЛЬЮ

6.1 Общая структура

Приложение построено по паттерну MVC (Model – View – Controller), который разделяет программу на три слоя: модель хранит данные и правила их обработки, представление отображает данные пользователю, контроллер обрабатывает действия пользователя. Схема взаимодействия слоев приведена на рисунке 10.

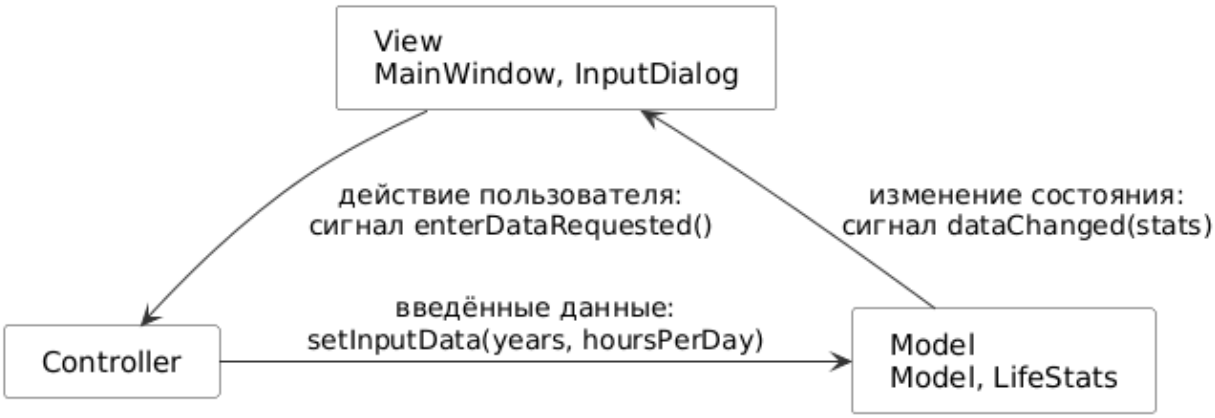


Рисунок 10 – Схема взаимодействия слоев MVC с активной моделью

Все три слоя создаются и связываются в одном месте – функции main() (файл main.cpp): создаются объекты Model, MainWindow и Controller, причем контроллер в своем конструкторе устанавливает обе связи – представления с контроллером и модели с представлением. Ни представление, ни модель при этом не знают о существовании друг друга: связь выполняется через сигналы и слоты, то есть по именам методов, а не по типам классов.

6.2 Слой Model

Слой модели не зависит от классов Qt Widgets и включает следующие элементы:

– LifeStats – структура состояния: принятые моделью значения (количество лет и часов в день) и результаты расчета;

– Model – активная модель: проверяет входные данные, хранит последнее принятое состояние, пересчитывает показатели и рассылает сигнал dataChanged;

– setInputData() – единственная точка входа в модель для контроллера: принимает данные, возвращает признак успеха и текст ошибки;

– recalculate() – закрытый метод расчета показателей.

Проверки корректности входных данных приведены в таблице 4. Проверка формата (является ли текст числом) выполняется контроллером при преобразовании строки, а проверка смысла значений – моделью.

Таблица 4 – Проверки корректности входных данных

Ситуация	Реакция программы
Поле количества лет не содержит целого числа	Сообщение «Введите корректные числовые значения», данные модели не изменяются
Поле количества часов не содержит числа	Сообщение «Введите корректные числовые значения», данные модели не изменяются
Количество лет вне диапазона от 1 до 120	Сообщение «Количество лет должно быть в диапазоне от 1 до 120»
Количество часов в день вне диапазона от 0 до 24	Сообщение «Количество часов в день должно быть в диапазоне от 0 до 24»
Данные корректны	Модель сохраняет значения, пересчитывает показатели и рассылает сигнал dataChanged

Расчет выполняется по усредненным статистическим данным о поведении пользователя персонального компьютера. Формулы приведены в таблице 5, где N – введенное количество лет, H – количество часов в день, T – общее время за компьютером в часах. Год принят равным 365,25 суток.

Таблица 5 – Показатели, вычисляемые программой

Показатель	Формула расчета
Время за компьютером, ч	$T = N \cdot 365{,}25 \cdot H$
Время за компьютером, сут	$T / 24$
Пробег курсора, км	$T \cdot 0{,}15$
Количество кликов мышью	$T \cdot 800$
Количество нажатий клавиш	$T \cdot 3000$

Коэффициенты вынесены в закрытые константы класса CURSOR_KM_PER_HOUR, CLICKS_PER_HOUR и KEYSTROKES_PER_HOUR, поэтому уточнение статистических данных не требует изменения остального кода.

6.3 Слой View

Слой представления состоит из двух классов:

– MainWindow – главная форма с кнопкой «Ввести данные», меткой последних введенных значений и группой «Результаты» из пяти строк;

– InputDialog – модальное окно ввода с полями количества лет и часов в день и кнопками OK и Cancel.

Главная форма ничего не вычисляет: в слоте onModelDataChanged() она только переносит значения из структуры LifeStats в метки. Нажатие кнопки преобразуется в сигнал enterDataRequested(), то есть представление сообщает о действии пользователя, не зная, кто и как его обработает.

Поля ввода сделаны текстовыми (QLineEdit), а не счетчиками (QSpinBox) намеренно: счетчик не позволил бы ввести некорректное значение, и проверка данных, которая по условию задания должна выполняться программой, оказалась бы лишней. Кроме того, проверка данных – обязанность модели, а не элементов интерфейса.

Восстановление последних введенных данных выполняется методом setInitialValues(): перед показом окна контроллер берет последнее состояние у модели и передает его в диалог, поэтому пользователю не требуется вводить данные заново.

6.4 Слой Controller

Контроллер Controller содержит слот onEnterDataRequested(), связанный с сигналом enterDataRequested() главной формы. Слот создает окно ввода, заполняет его последними данными из модели, показывает окно и, если пользователь подтвердил ввод, преобразует текст полей в числа методами QString::toInt() и QString::toDouble(). При неудачном преобразовании выводится сообщение об ошибке формата, иначе данные передаются модели методом setInputData(). Если модель отклоняет данные, контроллер показывает сообщение с текстом, сформированным моделью.

Контроллер не обновляет представление: после успешного вызова setInputData() он завершает работу, а главная форма обновляется по сигналу модели. Именно это и отличает активную модель от пассивной.

6.5 Взаимодействие компонентов

Последовательность обработки нажатия кнопки «Ввести данные» приведена на диаграмме последовательности (рисунок 11).

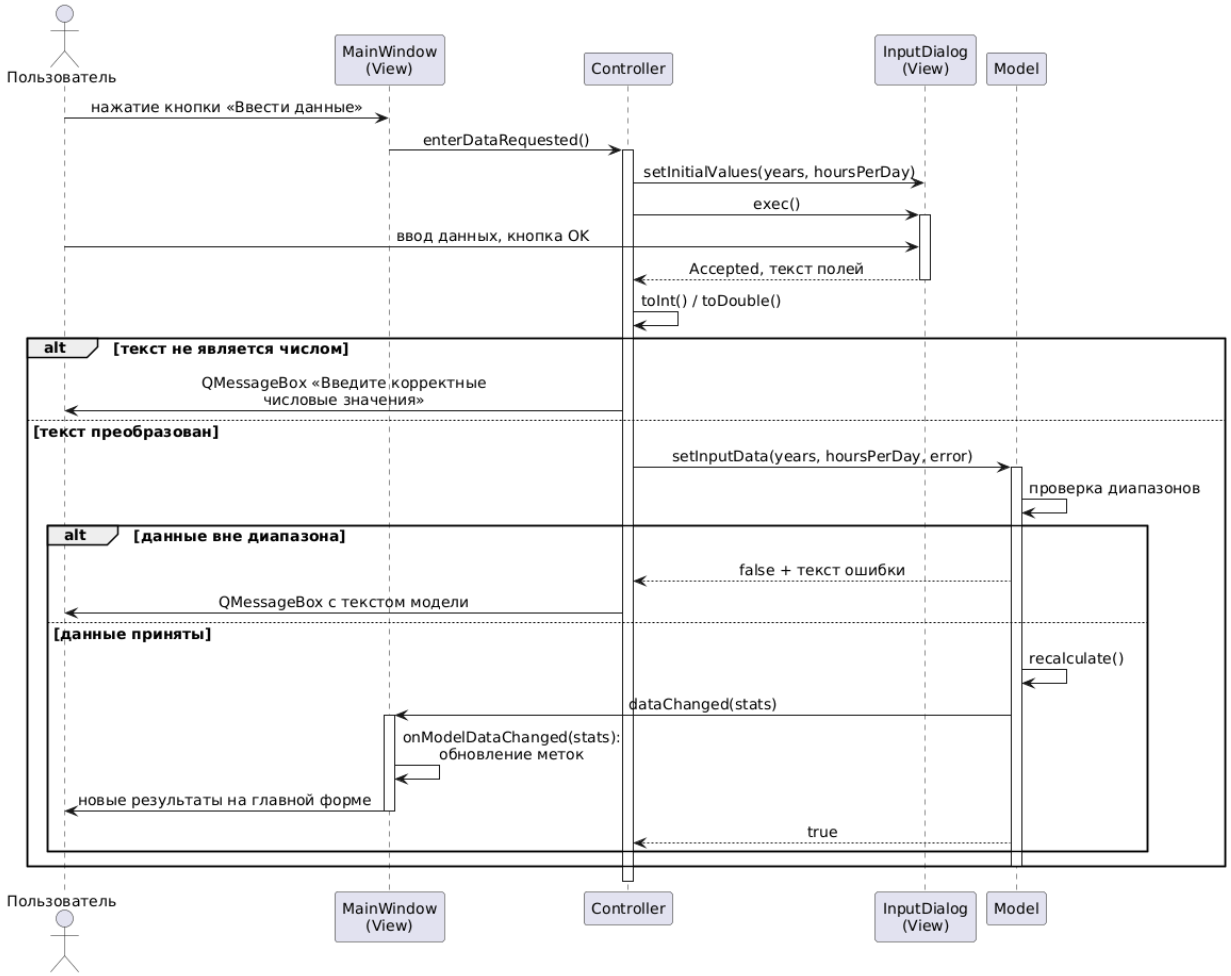


Рисунок 11 – Диаграмма последовательности обработки ввода данных

Пользователь нажимает кнопку, главная форма испускает сигнал и тем самым передает событие контроллеру. Контроллер получает у модели последнее состояние, заполняет им окно ввода и показывает его. После подтверждения ввода контроллер преобразует текст в числа и передает их модели. Модель проверяет значения, пересчитывает показатели и рассылает сигнал dataChanged, который принимает слот главной формы. Ветви alt на диаграмме показывают обработку ошибок: ошибку формата обнаруживает контроллер, ошибку диапазона – модель.

6.6 Отличие активной модели от пассивной

В схеме с пассивной моделью модель не знает о представлениях и не рассылает уведомлений: после вызова setInputData() контроллер должен сам запросить у модели новое состояние и передать его представлению, например вызовом view->onModelDataChanged(model->stats()). В активной модели такой

строки нет: модель сама рассылает сигнал dataChanged, на который подписано представление, и контроллер об обновлении интерфейса не заботится.

Активная модель удобнее, когда одно состояние отображается несколькими представлениями: новое представление достаточно подписать на сигнал модели, а контроллер при этом не меняется. Ее недостаток – необходимость механизма оповещения (в данной работе это сигналы и слоты Qt) и наследование модели от класса QObject.

6.7 Результаты работы программы

При запуске программы отображается главная форма с сообщением о том, что данные еще не введены, и пустыми полями результатов (рисунок 12).

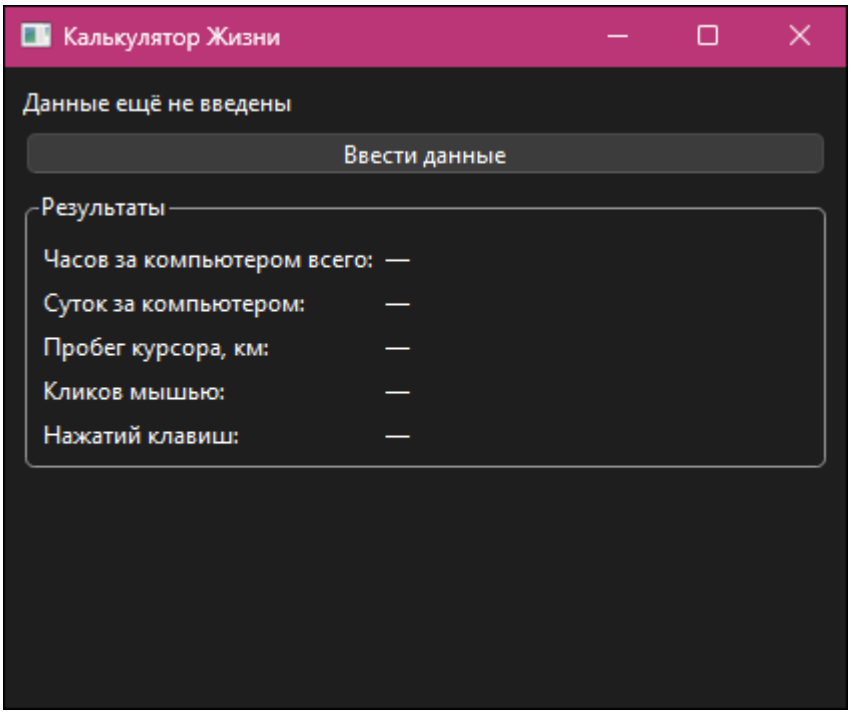


Рисунок 12 – Главная форма при запуске программы

По нажатию кнопки «Ввести данные» открывается отдельное окно ввода. При первом открытии его поля пусты (рисунок 13).

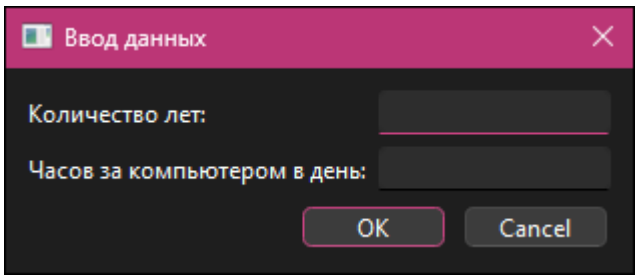


Рисунок 13 – Окно ввода данных при первом открытии

Для проверки работы программы введены количество лет 20 и количество часов в день 8 (рисунок 14).

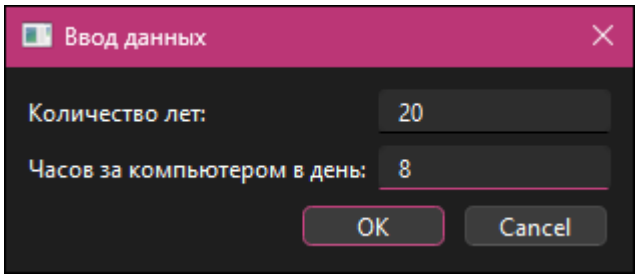


Рисунок 14 – Окно ввода с введенными данными

После подтверждения ввода главная форма отображает принятые значения и результаты расчета: 58 440 часов (2 435 суток) за компьютером, 8 766 км пробега курсора, 46 752 000 кликов мышью и 175 320 000 нажатий клавиш (рисунок 15). Обновление выполнено слотом главной формы по сигналу модели.

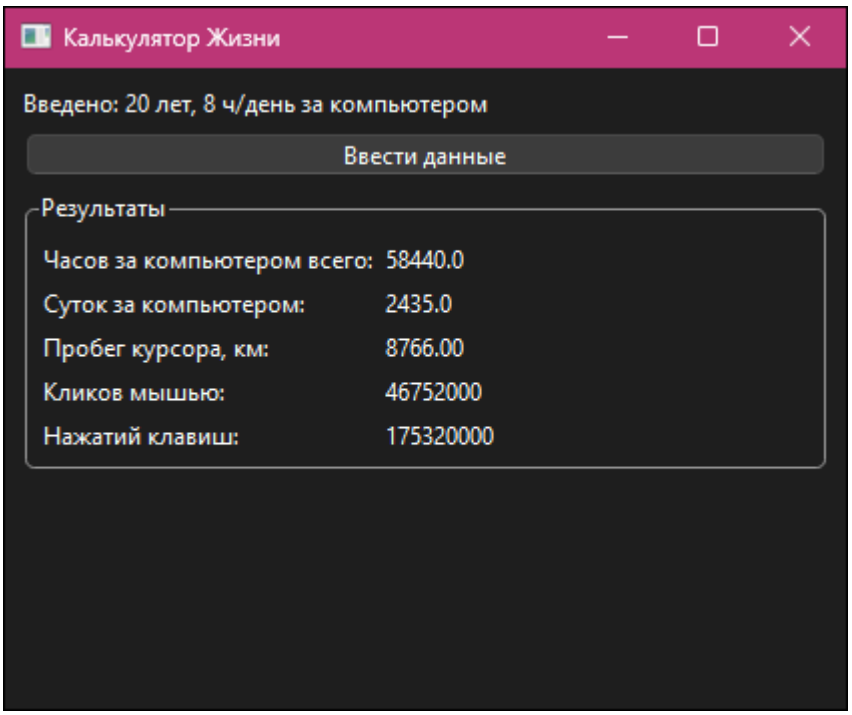


Рисунок 15 – Главная форма с результатами расчета

При повторном открытии окна ввода в поля подставлены последние введенные значения (рисунок 16).

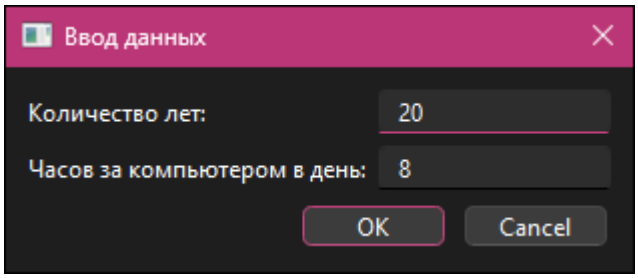


Рисунок 16 – Окно ввода с восстановленными значениями

Если в поле введен текст, не являющийся числом, преобразование в контроллере завершается неудачей и выводится сообщение об ошибке формата (рисунок 17). Состояние модели и главной формы при этом не изменяется.

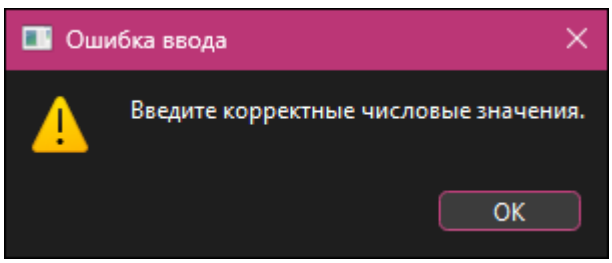


Рисунок 17 – Сообщение об ошибке при нечисловом вводе

Если значение является числом, но выходит за допустимые границы (например, 30 часов в день), его отклоняет уже модель, а контроллер показывает сформированное моделью сообщение (рисунок 18).

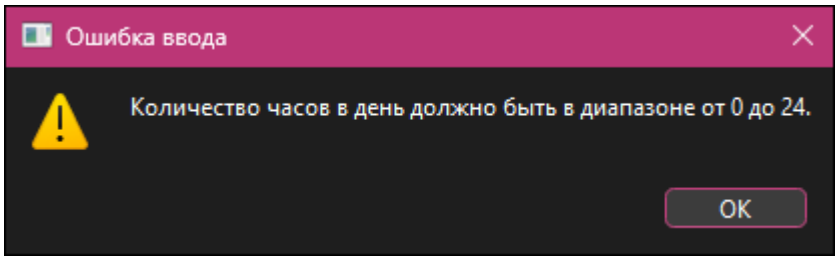


Рисунок 18 – Сообщение об ошибке при выходе значения за диапазон

6.8 Соответствие программы требованиям варианта

Соответствие разработанной программы требованиям варианта 14 приведено в таблице 6.

Таблица 6 – Выполнение требований варианта

Требование	Реализация
Ввод количества лет и часов в день	Поля окна InputDialog, значения передаются в виде текста
Кнопка «Ввести данные» на главной форме	Кнопка m_enterDataButton класса MainWindow
Отдельное окно для ввода данных	Модальный диалог InputDialog, открывается контроллером
Запоминание и восстановление последних данных	Состояние LifeStats в модели, метод InputDialog::setInitialValues()
Расчет показателей	Метод Model::recalculate()
Отображение результата на главной форме	Слот MainWindow::onModelDataChanged()
Сообщение об ошибке	QMessageBox::warning() в контроллере, текст ошибки диапазона формирует модель
Каркас MVC с активной моделью	Сигнал Model::dataChanged подключен напрямую к слоту главной формы

7 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1 Какие компоненты входят в состав IDE?

В состав интегрированной среды разработки входят редактор кода с подсветкой синтаксиса и автодополнением, компилятор или интерпретатор языка, система сборки, отладчик, средства управления проектом, средства рефакторинга и статического анализа, интеграция с системой управления версиями, диспетчер расширений, а во многих средах – и визуальный конструктор графического интерфейса.

2 Каковы основные приемы работы с редактором кода?

Основные приемы: автодополнение, переход к определению и объявлению символа (F12), поиск по символу и по файлам проекта, переключение между заголовочным файлом и файлом реализации, автоматическое форматирование и расстановка отступов, рефакторинг (переименование символа, извлечение метода), быстрые исправления по подсказкам компилятора и анализатора, сворачивание блоков кода, работа с закладками и множественным выделением.

3 Каким образом выполняется сборка и запуск проекта?

Проект описан файлом CMakeLists.txt, а параметры конфигурации – файлом CMakePresets.json. Среда вызывает CMake, который по этим файлам создает файлы сборки для генератора Ninja. Затем компилятор MSVC компилирует исходные тексты в объектные файлы, а компоновщик собирает из них исполняемый файл. Запуск выполняется из среды сочетанием Ctrl+F5 (без отладки) или клавишей F5 (под отладчиком). Те же действия доступны из командной строки командами cmake --preset x64-debug и cmake --build --preset x64-debug.

4 Что такое MVC?

MVC (Model – View – Controller) – архитектурный паттерн, разделяющий приложение на три слоя: модель хранит данные и бизнес-логику их обработки, представление отвечает за отображение данных пользователю, контроллер обрабатывает действия пользователя и преобразует их в вызовы модели. Разделение уменьшает связанность слоев, упрощает изменение интерфейса без изменения логики и позволяет тестировать модель отдельно от интерфейса.

5 Чем отличается активная модель от пассивной?

В пассивной модели модель не знает о представлениях и никого не оповещает: после изменения состояния представление обновляет контроллер,

явно запрашивая у модели новые данные. В активной модели модель сама оповещает подписанные на нее представления об изменении состояния по шаблону «Наблюдатель», а контроллер только передает ей входные данные. Активная модель удобнее при нескольких представлениях одного состояния, но требует механизма оповещения; пассивная проще в реализации, но возлагает синхронизацию интерфейса с данными на контроллер.

ВЫВОД

В ходе лабораторной работы выбрана система программирования: язык C++ со стандартом C++17, библиотека графического интерфейса Qt 6 и среда разработки Visual Studio Community 2026 с системой сборки CMake. Среда установлена и настроена, изучены ее функциональные возможности и средства отладки: работа отладчика показана на двух точках останова с просмотром значений переменных и стека вызовов.

Создан проект под вариант 14. Для работы с UML установлено расширение PlantUML Editor, диаграммы классов, последовательности и схема слоев хранятся в проекте в текстовой нотации PlantUML. Расширений SysML v2 для выбранной среды не существует, поэтому модель системы описана текстом в нотации SysML v2 и приведена в приложении Б.

Разработана программа «Калькулятор Жизни» по схеме MVC с активной моделью: модель самостоятельно проверяет данные, пересчитывает показатели и рассылает сигнал об изменении состояния, представление обновляется по этому сигналу, а контроллер только передает модели введенные пользователем данные. Все требования варианта выполнены, что подтверждается таблицей 6 и приведенными экранными формами.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1 Visual Studio Documentation [Электронный ресурс]. – Режим доступа : https://learn.microsoft.com/visualstudio/.

2 Qt 6 Documentation [Электронный ресурс]. – Режим доступа : https://doc.qt.io/qt-6/.

3 CMake Documentation [Электронный ресурс]. – Режим доступа : https://cmake.org/documentation/.

4 PlantUML [Электронный ресурс]. – Режим доступа : https://plantuml.com/.

5 OMG Systems Modeling Language (SysML) Version 2 [Электронный ресурс]. – Режим доступа : https://www.omg.org/spec/SysML/2.0/.

6 Приемы объектно-ориентированного проектирования. Паттерны проектирования / Э. Гамма [и др.]. – СПб. : Питер, 2020. – 448 с.

7 Жизненный цикл разработки программного обеспечения : лабораторная работа № 1 «Интегрированная среда разработки ПО». – Минск : БГУИР, 2026.

ПРИЛОЖЕНИЕ А

(обязательное)

Исходные тексты программы

Файл CMakeLists.txt

```
cmake_minimum_required(VERSION 3.16)

project(LifeCalculatorMVC LANGUAGES CXX)

set(CMAKE_AUTOMOC ON)
set(CMAKE_AUTORCC ON)
set(CMAKE_AUTOUIC ON)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

find_package(QT NAMES Qt6 Qt5 REQUIRED COMPONENTS Widgets)
find_package(Qt${QT_VERSION_MAJOR} REQUIRED COMPONENTS Widgets)

add_executable(LifeCalculatorMVC
    main.cpp
    model.h
    model.cpp
    mainwindow.h
    mainwindow.cpp
    inputdialog.h
    inputdialog.cpp
    controller.h
    controller.cpp
)

target_link_libraries(LifeCalculatorMVC
    PRIVATE Qt${QT_VERSION_MAJOR}::Widgets)

if(MSVC)
    target_compile_options(LifeCalculatorMVC PRIVATE /utf-8)
endif()
```

Файл CMakePresets.json

```
{
  "version": 3,
  "configurePresets": [
    {
      "name": "x64-debug",
      "displayName": "x64 Debug (MSVC, Qt 6.7.3)",
      "generator": "Ninja",
      "binaryDir": "${sourceDir}/build",
      "architecture": { "value": "x64", "strategy": "external" },
      "cacheVariables": {
        "CMAKE_BUILD_TYPE": "Debug",
        "CMAKE_PREFIX_PATH": "E:/Qt/6.7.3/msvc2019_64"
      }
```

```
    }
  ],
  "buildPresets": [
    { "name": "x64-debug", "configurePreset": "x64-debug" }
  ]
}
```

Файл main.cpp

```
#include <QApplication>

#include "controller.h"
#include "mainwindow.h"
#include "model.h"

int main(int argc, char *argv[])
{
    QApplication app(argc, argv);

    Model model;
    MainWindow view;
    Controller controller(&model, &view);

    view.resize(420, 320);
    view.show();

    return app.exec();
}
```

Файл model.h

```
#pragma once

#include <QObject>
#include <QString>

// Результаты расчёта "Калькулятора Жизни" за компьютером.
struct LifeStats
{
    int years = 0;
    double hoursPerDay = 0.0;

    double totalHours = 0.0;
    double totalDays = 0.0;
    double cursorMileageKm = 0.0;
    qint64 clicksCount = 0;
    qint64 keystrokesCount = 0;
};

// Активная модель: сама валидирует и пересчитывает данные,
// и уведомляет подписчиков (View) сигналом dataChanged об изменениях.
class Model : public QObject
{
    Q_OBJECT
public:
    explicit Model(QObject *parent = nullptr);
```

```
    // Проверяет данные и, если они корректны, обновляет состояние
    // и рассылает сигнал dataChanged. При ошибке возвращает false
    // и заполняет errorMessage.
    bool setInputData(int years, double hoursPerDay, QString &errorMessage);

    const LifeStats &stats() const { return m_stats; }

signals:
    void dataChanged(const LifeStats &stats);

private:
    // Усреднённые статистические коэффициенты для пользователя ПК:
    // дистанция перемещения курсора, клики и нажатия клавиш в час.
    static constexpr double DAYS_PER_YEAR = 365.25;
    static constexpr double CURSOR_KM_PER_HOUR = 0.15;
    static constexpr double CLICKS_PER_HOUR = 800.0;
    static constexpr double KEYSTROKES_PER_HOUR = 3000.0;

    LifeStats m_stats;

    void recalculate();
};
```

Файл model.cpp

```
#include "model.h"

Model::Model(QObject *parent) : QObject(parent)
{
}

bool Model::setInputData(int years, double hoursPerDay,
                         QString &errorMessage)
{
    if (years <= 0 || years > 120) {
        errorMessage = QStringLiteral(
            "Количество лет должно быть в диапазоне от 1 до 120.");
        return false;
    }
    if (hoursPerDay <= 0.0 || hoursPerDay > 24.0) {
        errorMessage = QStringLiteral(
            "Количество часов в день должно быть в диапазоне "
            "от 0 до 24.");
        return false;
    }

    m_stats.years = years;
    m_stats.hoursPerDay = hoursPerDay;
    recalculate();

    emit dataChanged(m_stats);
    return true;
}

void Model::recalculate()
{
```

```
    m_stats.totalHours =
        m_stats.years * DAYS_PER_YEAR * m_stats.hoursPerDay;
    m_stats.totalDays = m_stats.totalHours / 24.0;
    m_stats.cursorMileageKm = m_stats.totalHours * CURSOR_KM_PER_HOUR;
    m_stats.clicksCount =
        static_cast<qint64>(m_stats.totalHours * CLICKS_PER_HOUR);
    m_stats.keystrokesCount =
        static_cast<qint64>(m_stats.totalHours * KEYSTROKES_PER_HOUR);
}
```

Файл mainwindow.h

```
#pragma once

#include <QMainWindow>

#include "model.h"

class QLabel;
class QPushButton;

// Главное окно (View). Ничего не знает о Controller напрямую:
// сообщает о действии пользователя сигналом enterDataRequested
// и подписывается на изменения активной модели (onModelDataChanged).
class MainWindow : public QMainWindow
{
    Q_OBJECT
public:
    explicit MainWindow(QWidget *parent = nullptr);

signals:
    void enterDataRequested();

public slots:
    void onModelDataChanged(const LifeStats &stats);

private:
    QLabel *m_inputSummaryLabel;
    QLabel *m_totalHoursLabel;
    QLabel *m_totalDaysLabel;
    QLabel *m_cursorMileageLabel;
    QLabel *m_clicksLabel;
    QLabel *m_keystrokesLabel;
    QPushButton *m_enterDataButton;
};
```

Файл mainwindow.cpp

```
#include "mainwindow.h"

#include <QFormLayout>
#include <QGroupBox>
#include <QLabel>
#include <QPushButton>
#include <QVBoxLayout>
#include <QWidget>
```

```
MainWindow::MainWindow(QWidget *parent) : QMainWindow(parent)
{
    setWindowTitle(tr("Калькулятор Жизни"));

    m_inputSummaryLabel = new QLabel(tr("Данные ещё не введены"), this);
    m_totalHoursLabel = new QLabel(tr("—"), this);
    m_totalDaysLabel = new QLabel(tr("—"), this);
    m_cursorMileageLabel = new QLabel(tr("—"), this);
    m_clicksLabel = new QLabel(tr("—"), this);
    m_keystrokesLabel = new QLabel(tr("—"), this);
    m_enterDataButton = new QPushButton(tr("Ввести данные"), this);

    auto *resultsGroup = new QGroupBox(tr("Результаты"), this);
    auto *formLayout = new QFormLayout;
    formLayout->addRow(tr("Часов за компьютером всего:"),
                       m_totalHoursLabel);
    formLayout->addRow(tr("Суток за компьютером:"), m_totalDaysLabel);
    formLayout->addRow(tr("Пробег курсора, км:"), m_cursorMileageLabel);
    formLayout->addRow(tr("Кликов мышью:"), m_clicksLabel);
    formLayout->addRow(tr("Нажатий клавиш:"), m_keystrokesLabel);
    resultsGroup->setLayout(formLayout);

    auto *central = new QWidget(this);
    auto *mainLayout = new QVBoxLayout(central);
    mainLayout->addWidget(m_inputSummaryLabel);
    mainLayout->addWidget(m_enterDataButton);
    mainLayout->addWidget(resultsGroup);
    mainLayout->addStretch(1);
    setCentralWidget(central);

    connect(m_enterDataButton, &QPushButton::clicked,
            this, &MainWindow::enterDataRequested);
}

void MainWindow::onModelDataChanged(const LifeStats &stats)
{
    m_inputSummaryLabel->setText(
        tr("Введено: %1 лет, %2 ч/день за компьютером")
            .arg(stats.years)
            .arg(stats.hoursPerDay));

    m_totalHoursLabel->setText(QString::number(stats.totalHours, 'f', 1));
    m_totalDaysLabel->setText(QString::number(stats.totalDays, 'f', 1));
    m_cursorMileageLabel->setText(
        QString::number(stats.cursorMileageKm, 'f', 2));
    m_clicksLabel->setText(QString::number(stats.clicksCount));
    m_keystrokesLabel->setText(QString::number(stats.keystrokesCount));
}
```

Файл inputdialog.h

```
#pragma once

#include <QDialog>

class QLineEdit;
```

```
// Диалоговое окно ввода данных (элемент View).
class InputDialog : public QDialog
{
    Q_OBJECT
public:
    explicit InputDialog(QWidget *parent = nullptr);

    // Предзаполняет поля последними введёнными значениями.
    void setInitialValues(int years, double hoursPerDay);

    QString yearsText() const;
    QString hoursPerDayText() const;

private:
    QLineEdit *m_yearsEdit;
    QLineEdit *m_hoursEdit;
};
```

Файл inputdialog.cpp

```
#include "inputdialog.h"

#include <QDialogButtonBox>
#include <QFormLayout>
#include <QLineEdit>
#include <QVBoxLayout>

InputDialog::InputDialog(QWidget *parent) : QDialog(parent)
{
    setWindowTitle(tr("Ввод данных"));

    m_yearsEdit = new QLineEdit(this);
    m_hoursEdit = new QLineEdit(this);

    auto *formLayout = new QFormLayout;
    formLayout->addRow(tr("Количество лет:"), m_yearsEdit);
    formLayout->addRow(tr("Часов за компьютером в день:"), m_hoursEdit);

    auto *buttonBox = new QDialogButtonBox(
        QDialogButtonBox::Ok | QDialogButtonBox::Cancel, this);
    connect(buttonBox, &QDialogButtonBox::accepted, this, &QDialog::accept);
    connect(buttonBox, &QDialogButtonBox::rejected, this, &QDialog::reject);

    auto *mainLayout = new QVBoxLayout(this);
    mainLayout->addLayout(formLayout);
    mainLayout->addWidget(buttonBox);
}

void InputDialog::setInitialValues(int years, double hoursPerDay)
{
    if (years > 0)
        m_yearsEdit->setText(QString::number(years));
    if (hoursPerDay > 0)
        m_hoursEdit->setText(QString::number(hoursPerDay));
}

QString InputDialog::yearsText() const
```

```
{
    return m_yearsEdit->text();
}

QString InputDialog::hoursPerDayText() const
{
    return m_hoursEdit->text();
}
```

Файл controller.h

```
#pragma once

#include <QObject>

class Model;
class MainWindow;

// Controller связывает View и Model: реагирует на действия пользователя,
// передаёт "сырые" данные модели и показывает сообщения об ошибках.
// Обновление View выполняет сама Model через сигнал dataChanged
// (активная модель).
class Controller : public QObject
{
    Q_OBJECT
public:
    Controller(Model *model, MainWindow *view, QObject *parent = nullptr);

private slots:
    void onEnterDataRequested();

private:
    Model *m_model;
    MainWindow *m_view;
};
```

Файл controller.cpp

```
#include "controller.h"

#include <QMessageBox>

#include "inputdialog.h"
#include "mainwindow.h"
#include "model.h"

Controller::Controller(Model *model, MainWindow *view, QObject *parent)
    : QObject(parent), m_model(model), m_view(view)
{
    connect(m_view, &MainWindow::enterDataRequested,
            this, &Controller::onEnterDataRequested);
    connect(m_model, &Model::dataChanged,
            m_view, &MainWindow::onModelDataChanged);
}

void Controller::onEnterDataRequested()
```

```
{
    InputDialog dialog(m_view);
    const LifeStats &current = m_model->stats();
    dialog.setInitialValues(current.years, current.hoursPerDay);

    if (dialog.exec() != QDialog::Accepted)
        return;

    bool yearsOk = false;
    bool hoursOk = false;
    const int years = dialog.yearsText().toInt(&yearsOk);
    const double hours = dialog.hoursPerDayText().toDouble(&hoursOk);

    if (!yearsOk || !hoursOk) {
        QMessageBox::warning(m_view, tr("Ошибка ввода"),
                             tr("Введите корректные числовые значения."));
        return;
    }

    QString errorMessage;
    if (!m_model->setInputData(years, hours, errorMessage)) {
        QMessageBox::warning(m_view, tr("Ошибка ввода"), errorMessage);
    }
}
```

ПРИЛОЖЕНИЕ Б

(обязательное)

Модель системы в нотации SysML v2

Файл docs/sysmlv2-model.sysml

```
package LifeCalculator {

    doc /*
     * Модель системы «Калькулятор Жизни» (вариант 14)
     * в текстовой нотации SysML v2.
     */

    // --- Типы данных ----------------------------------------------------

    attribute def RawLifeInput {
        doc /* «Сырые» данные из окна ввода: хранятся строками, чтобы
             * проверку выполняла модель, а не элементы интерфейса. */
        attribute yearsText : String;
        attribute hoursPerDayText : String;
    }

    attribute def LifeStats {
        doc /* Принятые моделью данные и результаты расчёта. */
        attribute years : Integer;
        attribute hoursPerDay : Real;
        attribute totalHours : Real;
        attribute totalDays : Real;
        attribute cursorMileageKm : Real;
        attribute clicksCount : Integer;
        attribute keystrokesCount : Integer;
    }

    // --- Структура системы ----------------------------------------------

    part def ModelLayer {
        attribute stats : LifeStats;
        port notify : NotificationPort;
    }

    part def ViewLayer {
        port observe : ~NotificationPort;
    }

    part def ControllerLayer;

    port def NotificationPort {
        out item dataChanged : LifeStats;
    }

    part def Application {
        part model : ModelLayer;
        part view : ViewLayer;
        part controller : ControllerLayer;
```

```
        // Активная модель: уведомление идёт от модели к представлению
        // напрямую, минуя контроллер.
        connect model.notify to view.observe;
    }

    // --- Поведение ---------------------------------------------------------

    action def CalculateLifeStats {
        in raw : RawLifeInput;
        out stats : LifeStats;

        action remember;      // запомнить последний ввод
        action validate;      // проверить диапазоны
        action recalculate;   // пересчитать показатели
        action notify;        // разослать уведомление наблюдателям

        first remember then validate;
        succession validate then recalculate;
        succession recalculate then notify;
    }

    // --- Требования варианта 14 --------------------------------------------

    requirement def <R1> EnterDataButton {
        doc /* На главной форме размещена кнопка «Ввести данные». */
    }

    requirement def <R2> SeparateInputWindow {
        doc /* Ввод данных выполняется в отдельном окне. */
    }

    requirement def <R3> RestoreLastInput {
        doc /* Последние введённые данные запоминаются и восстанавливаются
             * при повторном открытии окна ввода. */
    }

    requirement def <R4> ShowResults {
        doc /* Результаты расчёта отображаются на главной форме. */
    }

    requirement def <R5> ErrorMessage {
        doc /* При некорректных данных выводится сообщение об ошибке. */
    }

    requirement def <R6> ActiveModelMvc {
        doc /* Реализован каркас MVC с активной моделью. */
    }

    part variant14 : Application {
        satisfy requirement :>> EnterDataButton;
        satisfy requirement :>> SeparateInputWindow;
        satisfy requirement :>> RestoreLastInput;
        satisfy requirement :>> ShowResults;
        satisfy requirement :>> ErrorMessage;
        satisfy requirement :>> ActiveModelMvc;
    }
}
```